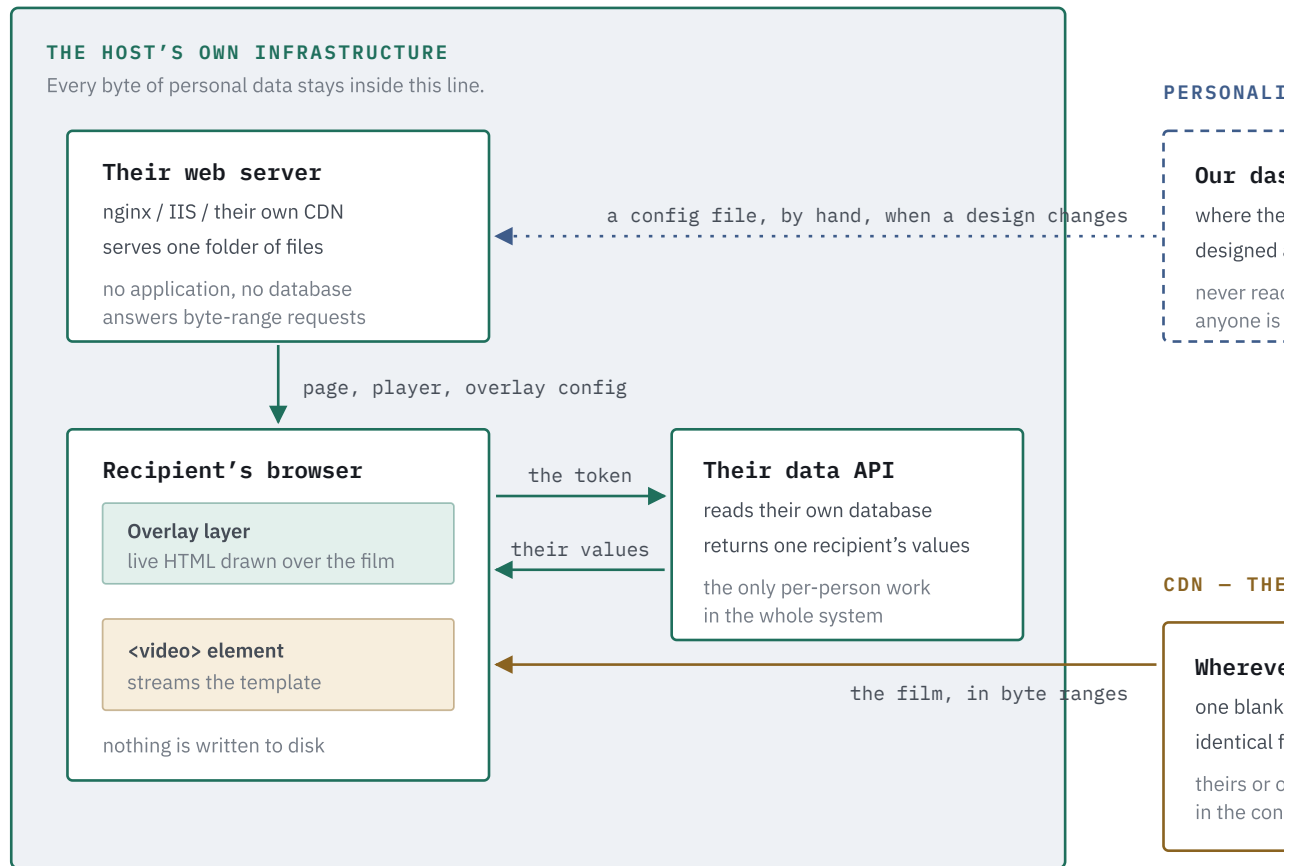


Running the player on someone else's infrastructure

How a personalised video statement is delivered from a client's own servers, with no request reaching Personaliz while a recipient is watching.

The film is a static file. The personalisation is a small JSON file. The browser puts them together.

Nothing is rendered per recipient, nothing is stored, and no service of ours sits in the path. What the client hosts is a folder; what they supply is one lookup against their own data.



The boundary is the argument. Every recipient value, and every request carrying one, stays inside the enclosure. Two lines cross it and neither carries personal data: the dotted one is a file handed over when a design changes, not a connection; the solid one fetches the same blank film every viewer gets, addressed by a plain URL. Point that URL at the host's own storage and nothing crosses the boundary at all.

■ On the host's infrastructure ■ Outside, contacted during playback — the film, nothing else

■ Outside, contacted only at design time

TWO PHASES, DELIBERATELY SEPARATE

Most of the confusion about this architecture comes from collapsing these two. Our dashboard is involved in one of them and absent from the other.

Design ONCE, UP FRONT

Someone positions the elements over the film, sets the timings and the animation.

- Happens in our dashboard, on our infrastructure
- Produces one config file per campaign
- Sent to the host the way any file is sent
- Repeats only when a design changes

Playback EVERY VIEW

A recipient opens their link and the browser assembles the statement.

- Host's web server hands over the folder
- Host's API returns that recipient's values
- Browser draws the overlay over the film
- Personaliz is not contacted at all

WHAT GETS DEPLOYED

One folder, and it is mostly the player. The figures are measured from the current build: two campaigns, an Arabic film with an 87-element overlay and an English one with 61. Each is a self-contained campaign reached by its own link.

FILE	WHAT IT IS	SIZE
index.html	An empty shell. No recipient data, no film, no text — everything arrives after load.	12 KB
_next/	The player itself: overlay engine, animation, variable substitution.	1.8 MB
video_d53f2d06.json	Arabic campaign. Positions, timings, keyframes. Identical for every recipient.	68 KB
video_28bf7be5.json	English campaign, its own film and its own overlay.	80 KB
contact1.json	One sample recipient. The host replaces this with their own lookup.	4 KB
images/ fonts/ chrome/	Overlay artwork, the Arabic typeface, and the player's own icons — all local rather than fetched.	1.5 MB
total	No runtime, no database, nothing to install. Films stream from the CDN.	3.6 MB
media/	Optional local copies of both films, for a deployment that wants no external origin at all.	+22 MB

WHAT WAS TAKEN OUT, AND WHY

The stock player is built for our own hosting. Five things had to go before it could run somewhere that must not reach us.

The first-load API call	Returned the film, the design, the recipient's values and a session in one response. Split into two files the host serves themselves.
Server-side page metadata	Fetches a link-preview thumbnail from us on every view, and forced the page to be rendered per request. Removing it is what makes a static build possible.
Device fingerprinting	Identified the browser. Not something a government host should be sending anywhere.
Geo-IP lookup	Sent the viewer's address to a third party.
Fonts and icons from our CDN	The Arabic typeface came from Google and the player's icons from our CloudFront — a third-party request on every view. Both now ship in the folder.

None of this rests on having found and deleted every call. A guard sits in front of every outgoing request, refuses anything not addressed to the host's own server, and records what it refused — so the claim can be checked rather than trusted. A Content-Security-Policy header on the host's server is the enforcement behind it, since a browser fetching an image or a film never passes through the guard at all.

WHAT THE HOST HAS TO PROVIDE

■ **A web server that answers byte-range requests.**

nginx, IIS, Apache and S3 with a CDN all do by default. This is what lets the film stream instead of downloading whole.

■ **Somewhere to put the folder.**

Static hosting. No Node process, no database, no install.



One API
returning
a
recipient's
values.

It receives the token already in the play link and returns a set of names and values. This is the only integration point, and it is one function in one file.

Everything else — the animation, the charts, the conditional sections, the right-to-left layout — runs in the recipient's browser from files the host already has.

MEASURED, NOT ASSERTED

Both campaigns were built with no configuration file present, exactly as a fresh clone of the repository would be, served from a plain static file server and driven in a real browser.

CURRENT BUILD		
Arabic	film decodes at 1920×1080, overlay draws throughout	0 failed requests
English	film decodes at 1024×576, overlay draws throughout	0 failed requests
Origins	Two, both expected: the host's own server, and the CDN the film is addressed from. Nothing else was contacted.	

One known gap: the English master is 1024×576 against the Arabic 1920×1080, and it has text baked into the film, so the difference is visible. Replacing it is an upload, not a code change.

Figures measured from the current build of the on-premise player — MOHRE establishment statement, Arabic and English — served from a static file server with no application behind it. Each language is a standalone campaign addressed by its own link; there is no in-player language switch.